Reviewer Pack — Fourier Coefficient Sum Bounds Monotone DNF Size for k-CLIQU…

Entry fd209c52f373 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 06:02:15 UTC

Fourier Coefficient Sum Bounds Monotone DNF Size for k-CLIQUE

Entry ID: `fd209c52f373` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 06:02:15 UTC

1. Conjecture

Field A (mathematical branch): Fourier Analysis on Boolean Functions **Field B** (complexity object): Monotone DNF Size for k-CLIQUE

Statement:

> For any monotone DNF formula on n variables, the sum of absolute values of its Fourier coefficients is O(log n). For the k-CLIQUE indicator function, this sum is Ω(n) for k ≥ 3.

Rationale (proposer's reasoning):

> Fourier coefficients capture the influence of variables in DNF formulas. Monotone k-CLIQUE requires exponentially many terms, leading to large coefficient sums, while polynomial-size DNFs have logarithmic sums due to sparsity.

Taxonomy category: `FOURIER_ANALYTIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `24d75763e20df1ea`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def walsh_hadamard_transform(f, n):
        if n == 1:
            return f[0]
        even = walsh_hadamard_transform([f[i] for i in range(0, len(f), 2)], n // 2)
        odd = walsh_hadamard_transform([f[i] for i in range(1, len(f), 2)], n // 2)
        return [even[i] + odd[i] for i in range(n // 2)] + [even[i] - odd[i] for i in range(n // 2

    def k_clique_indicator(n, k):
        if k >= n:
            return 1
        return sum(1 for subset in itertools.combinations(range(n), k) if len(set(subset)) == k)

    def generate_random_3cnf(n, m):
        clauses = []
        variables = set()
        for _ in range(m):
            clause = random.sample(range(-n, 0), 2) + [random.randint(1, n)]
            clauses.append(clause)
            variables.update(abs(lit) for lit in clause)
        return clauses, list(variables)

    def evaluate_dnf(dnf, assignment):
        for clause in dnf:
            if all(assignment[abs(lit) - 1] == (lit > 0) for lit in clause):
                return 1
        return 0

    n = random.randint(5, 40)
    m = random.randint(2 * n, 4 * n)
    dnf, variables = generate_random_3cnf(n, m)

    assignment = [random.choice([True, False]) for _ in range(n)]
    metric_value = abs(evaluate_dnf(dnf, assignment))

    if len(variables) == n:
        k_clique_sum = k_clique_indicator(n, 3)
        if k_clique_sum < 0.1 * n:
            return {
                "metric_name": "k-clique sum",
```

```
                "metric_value": k_clique_sum,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": "k-CLIQUE sum too small"
            }

    if metric_value > 10 * math.log(n):
        return {
            "metric_name": "Fourier coefficient sum",
            "metric_value": metric_value,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "Fourier coefficient sum too large"
        }

    return {
        "metric_name": "Fourier coefficient sum",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        for r in results:
            if not r["conjecture_holds"]:
                counterexample = r["counterexample"]
                first_failing_seed = seed
                break

        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)


```
', 'metric_value': 1, 'instances_tested': 1, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Fourier coefficient sum', 'metric_value': 1, 'instances_tested': 1, 'conjectu
TRIAL: {'metric_name': 'Fourier coefficient sum', 'metric_value': 1, 'instances_tested': 1, 'conjectu
TRIAL: {'metric_name': 'Fourier coefficient sum', 'metric_value': 1, 'instances_tested': 1, 'conjectu
TRIAL: {'metric_name': 'Fourier coefficient sum', 'metric_value': 1, 'instances_tested': 1, 'conjectu
TRIAL: {'metric_name': 'Fourier coefficient sum', 'metric_value': 1, 'instances_tested': 1, 'conjectu
TRIAL: {'metric_name': 'Fourier coefficient sum', 'metric_value': 1, 'instances_tested': 1, 'conjectu
TRIAL: {'metric_name': 'Fourier coefficient sum', 'metric_value': 1, 'instances_tested': 1, 'conjectu
TRIAL: {'metric_name': 'Fourier coefficient sum', 'metric_value': 1, 'instances_tested': 1, 'conjectu
TRIAL: {'metric_name': 'Fourier coefficient sum', 'metric_value': 1, 'instances_tested': 1, 'conjectu
TRIAL: {'metric_name': 'Fourier coefficient sum', 'metric_value': 1, 'instances_tested': 1, 'conjectu
TRIAL: {'metric_name': 'Fourier coefficient sum', 'metric_value': 1, 'instances_tested': 1, 'conjectu
TRIAL: {'metric_name': 'Fourier coefficient sum', 'metric_value': 1, 'instances_tested': 1, 'conjectu
TRIAL: {'metric_name': 'Fourier coefficient sum', 'metric_value': 1, 'instances_tested': 1, 'conjectu
RESULT: SUPPORTED mean=0.9333333333333333 std=0.24944382578492943 support_fraction=1.0
```


8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> All trials use n=1 instances, making the Fourier coefficient sum trivially 1. This violates 'n too small' and 'metric saturation' failure modes. The conjecture's O(log n) bound becomes vacuous for n=1, and the Ω(n) claim for k-CLIQUE is impossible with n=1. The test lacks variation to detect scaling behavior.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> All trials used n=1, making the Fourier coefficient sum trivially 1. This violates scaling requirements for both O(log n) and Ω(n) claims. | next: Test with n ≥ 1000 and varying k to observe scaling behavior of Fourier coefficient sums

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	135946
2	propose	ollama_remote	qwen3:8b	0	0	73577
3	preregistration	ollama_remote	qwen3:8b	0	0	35096
4	novelty	ollama_remote	qwen3:8b	0	0	27703
5	novelty	ollama_remote	qwen3:8b	0	0	30158
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13962
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8443
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9409
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11467
10	critic	ollama_remote	qwen3:8b	0	0	28088
11	judge	ollama_remote	qwen3:8b	0	0	25025

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 398874 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/fd209c52f373.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fd209c52f373.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/fd209c52f373.tar.gz` (if generated)